

Tc3_XPlanarUtility

Introduction

Architecture



API

FB_TcloXPlanarEnvironment

FB_TcloXPlanarMovementArea

DRAFT | Document in Progress

FB_TcloXPlanarProcessingUnit

I_TcloXPlanarProcessingUnit

DRAFT | Document in Progress

 Methods

DRAFT | Document in Progress

	Name	Rückgabotyp	Zugriff	Definitionsort	Beschreibung
	GetObjectName	STRING(31)			Gibt die Bezeichnung des TcCom Objects zurück, die im System Manager definiert wurde.
Parameter (Init)					
01_General					
	GetOperationMode	OperationMode			In simulation mode no hardware is needed.
	GetTraceLevel	DINT			Controls the amount of log messages.
	GetWriteMiLog	DINT			TRUE: Write a log file for start up of ML-Modules.
02_MoverDetection					
	SetMoverDetectionSetup	BOOL			Can be adjusted if e.g. configured movers are not on the system and should not be detected.
	SetStartMoverRedetection	BOOL			Activates the mover detection during runtime. Can only be activated if all movers are deactivated.
03_ProcessingUnit	SetStartMoverIdentification	BOOL			Starts the identification and rearrangement of all movers on the system.
	GetEtherCatMasterSyncTaskObjectId	OTCID			This is the task whose EtherCAT master is the EtherCAT DC (distributed clocks) master.
Parameter (Online)					
00_ProcessingUnit					
	GetTaskCount	UDINT			Returns the number of tasks.
	GetTaskOids	BOOL			Returns an array with the task object IDs.
	GetDriverVersion	STRING			Returns the software version of the XPlanar driver.
10_Parts					
	GetPartCount	UDINT			Returns the number of parts.
	GetPartOids	BOOL			Returns an array containing the object IDs of all parts.
20_Tiles	GetTileCount	UDINT			Returns the number of tiles.
	GetTileOids	BOOL			Returns an array containing the object IDs of all tiles.
30_MovementAreas					

	Name	Rückgabotyp	Zugriff	Definitionsort	Beschreibung
	GetmovementAreaCount	UDINT			Returns the number of movement areas.
	GetMovementAreaOids	BOOL			Returns an array containing the object IDs of all parts.
40_Movers	GetMoverCount	UDINT			Returns the number of movers.
	GetMoverOids	BOOL			Returns an array containing the object IDs of all movers.
	GetMoverDetectionInfos	REFERENCE TO ST_TcloMoverDetectionInfos			Expected and detected mover count of each different movertype.
00_Subitems	GetMoverIdentificationStatus	REFERENCE TO MoverIdentificationStatus			
	MovementAreaTclo	I_TcloXPlanarMovementArea			
	MoverNc	I_Nc3PlanarMover			
	MoverTclo	I_TcloXPlanarMover			
	PartTclo	I_TcloXPlanarPart			
	TileTclo	I_TcloXPlanarTile			
	TileCoE	I_Aps1003Drive			

FB_TcloXPlanarMover

I_TcloXPlanarMover

 Methods

Name	Rückgabety	Zugriff	Definitionsort	Beschreibung
GetActualPositon	MoverPositionInfo		I_TcloXPlanarMover	
GetBtn	STRING(13)		I_TcloXPlanarMover	
GetComObject	BOOL		I_TcloXPlanarMover	
GetNcMoverOid	OTCID		I_TcloXPlanarMover	
GetZPosAtEnable	LREAL		I_TcloXPlanarMover	
GetObjectName	STRING(20)		I_TcloXPlanarMover	
GetPayload	ST_TcloPayload		I_TcloXPlanarMover	
SetPayload	BOOL		I_TcloXPlanarMover	
GetTaskOid	BOOL		I_TcloXPlanarMover	
GetMoverType				
GetParameterizationSetId				
GetPayloadInertia				
SetMoverType				
SetParameterizationSetId				
SetPayloadInertia				
SetForceLimitMax				
SetForceLimitMin				
SetControllersOn				
SetEnterOrLeaveMovementArea				
SetSwitchMovementArea	MoverVector			

 Eigenschaften

Name	Typ	Zugriff	Definitionsort	Beschreibung
ScopeAdr	I_XPlanarMoverScopeAdr	Get		

I_Nc3PlanarMover

Methods

Name	Rückgabotyp	Zugriff	Definitionsart	Beschreibung
Init			I_	
GetObjectName				
GetMoverHeight	LREAL			
GetMoverWidth	LREAL			
GetSimulationMode	BOOL			
GetMaximalAcceleration	MoverVector			
GetMaximalJerk	MoverVector			
GetMaximalVelocity	MoverVector			
GetMaximumLagFilterRime	MoverVector			
GetMaximumPositionLagValue	MoverVector			
GetPositionLagMonitoringEnabled	BOOL			
GetEnableState	MC_ENABLE_STATE			
GetMoverState	MC_MOVER_STATE			
GetPosition	MoverVector			

FB_TcloMoverParametrizationSet

DRAFT | Document in Progress

I_XPlanarMoverScopeAdr

🔗 Methods

Name	Rückgabotyp	Zugriff	Definitionsart	Beschreibung
GetActualPositions	POINTER TO MoverVector			
GetPositionError	POINTER TO MoverVector			
GetActualVelocities	POINTER TO MoverVector			
GetActualAccelerations	POINTER TO MoverVector			
GetExternalPositionSetpoint	POINTER TO MoverVector			
GetExternalVelocitySetpoints	POINTER TO MoverVector			
GetExternalAccelerationSetpoints	POINTER TO MoverVector			
GetInterpolatedPositionSetpoints	POINTER TO MoverVector			
GetInterpolatedVelocitySetpoints	POINTER TO MoverVector			
GetInterpolatedAccelerationSetpoints	POINTER TO MoverVector			
GetOutputForce	POINTER TO MoverVector			
GetLimitedOutputForces				
GetEstimatedMoverPower				
GetTilePowerLimitation				
GetPartOID				
GetMovementAreaOID				

FB_TcloXPlanarTile

I_TcloXPlanarTile

 Methods

Name	Rückgabotyp	Zugriff	Definitionsor	Beschreibung
GetTaskOid	OTCID		I_TcloXPlanarTile	
SetTaskOid	BOOL		I_TcloXPlanarTile	
GetBtn	STRING(13)		I_TcloXPlanarTile	
GetComObject	BOOL		I_TcloXPlanarTile	
GetObjectName	STRING(20)		I_TcloXPlanarTile	

 Eigenschaften

Name	Typ	Zugriff	Definitionso	Beschreibung
CoE	I_Aps1003Drive	Get		

CoE Parameter

I_Aps1003Drive

 Properties

Name	Typ	Zugriff	Definitionsort	Beschreibung
BtnData	I_Aps1003Btn			
DiagData	I_Aps1003DriveDiagData			
HwVersionData	I_Aps1003HwVersion			
SwVersionData	I_Aps1003SwVersion			
AmsNetId	T_AMSNETID			
SlaveAdr	UINT			

I_Aps1003Btn

 Eigenschaften

Name	Typ	Zugriff	Definitionsort	Beschreibung
Update	BOOL			
Btn	STRING(13)			

I_Aps1003DriveDiagData

 Properties

Name	Typ	Zugriff	Definitionsort	Beschreibung
Update				
All				
DcLinkVoltage				
TemperatureNorth				
TemperatureEast				
TemperatureSouth				
TemperatureWest				
TemperatureCenter				
PowerSupply Temperature				
FanRpm				
FanDutyCycle				
SupplyVoltage				
AuxiliaryVoltage				
AccelerationX				
AccelerationY				
AccelerationZ				
TimeToShutdown				
FbHwRev				

I_Aps1003HwVersion

 Properties

Name	Typ	Zugriff	Definitionsort	Beschreibung
Update	BOOL			
HwVersion	STRING(13)			

I_Aps1003SwVersion

 Properties

Name	Typ	Zugriff	Definitionsort	Beschreibung
Update	BOOL			
SwVersion	STRING(13)			

FB_TcloXPlanarPart

DRAFT | Document in Progress

FB_MCPlanarGroup

FB_McPlanarMover

FB_McPlanarTrack

DRAFT | Document in Progress

Initialization

```

1  PROGRAM MAIN
2  VAR
3      fbEnvironment      : FB_TcIoXPlanarEnvironment;
4      fbVisuXPlanarDiag  : FB_VisuXPlanarDiag;
5      nStateInitXpu      : INT;
6      ipXpu              : I_TcIoXPlanarProcessingUnit;
7
8  CASE nStateInitXpu OF
9  0: // init environment
10     IF fbEnvironment.Init(TRUE) THEN
11         fbEnvironment.Init(FALSE);
12         nStateInitXpu:=1;
13     END_IF
14
15 1: // init Visu
16     fbVisuXPlanarDiag.Environment := fbEnvironment;
17     nStateInitXpu := 3;
18
19 3:
20     ipXpu := fbEnvironment.XpuTcIo(1);
21     nStateInitXpu := 4;
22
23 ELSE // call Visu
24     fbVisuXPlanarDiag.Cycle();
25 END_CASE

```

```

VAR
    fbEnvironment      : FB_TcIoXPlanarEnvironment;
    ipXpu              : I_TcIoXPlanarProcessingUnit;
    fbXPlanarUtilityVisu : FB_XPlanarUtilityVisu;
    nStateInitXpu      : INT;
END_VAR

CASE nStateInitXpu OF
0: // init environment
    IF fbEnvironment.Init(TRUE) THEN
        fbEnvironment.Init(FALSE);
        nStateInitXpu:=1;
    END_IF

1: // init Visu
    fbXPlanarUtilityVisu.Environment := fbEnvironment;
    nStateInitXpu := 3;

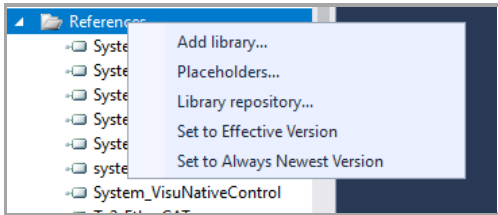
3:
    ipXpu := fbEnvironment.XpuTcIo(1);
    nStateInitXpu := 4;

ELSE // call Visu
    fbXPlanarUtilityVisu.Cycle();
END_CASE

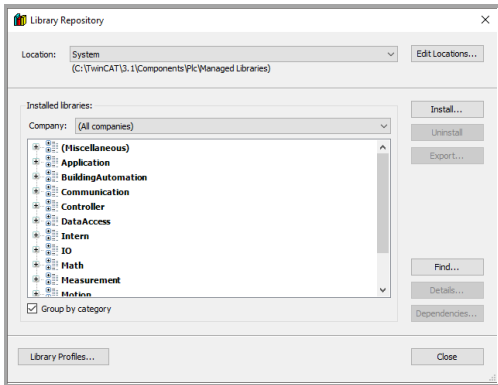
```

Visualization

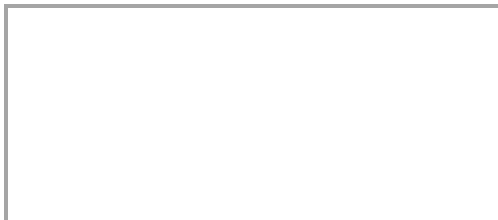
Install library



- ▶ References
- ▶ Library repository

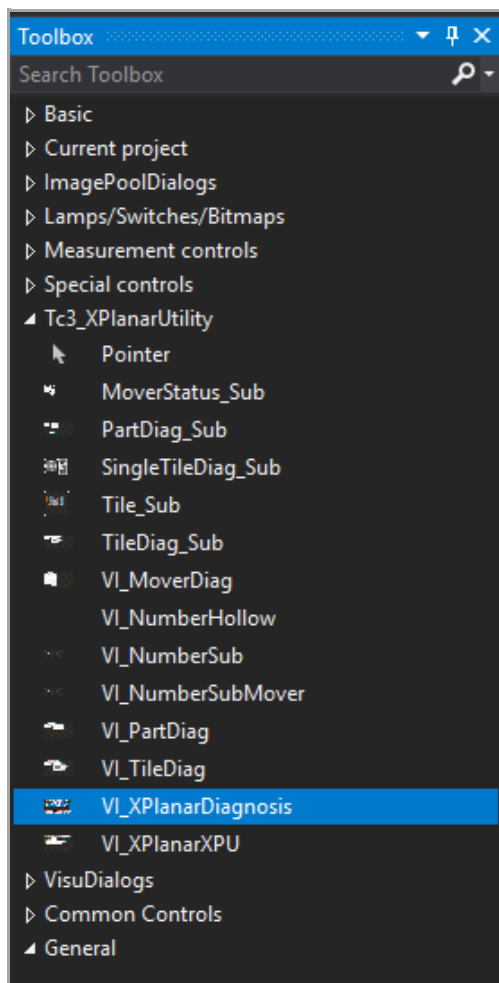


- ▶ Install
- ▶ Select the library file
- ▶ Open the file

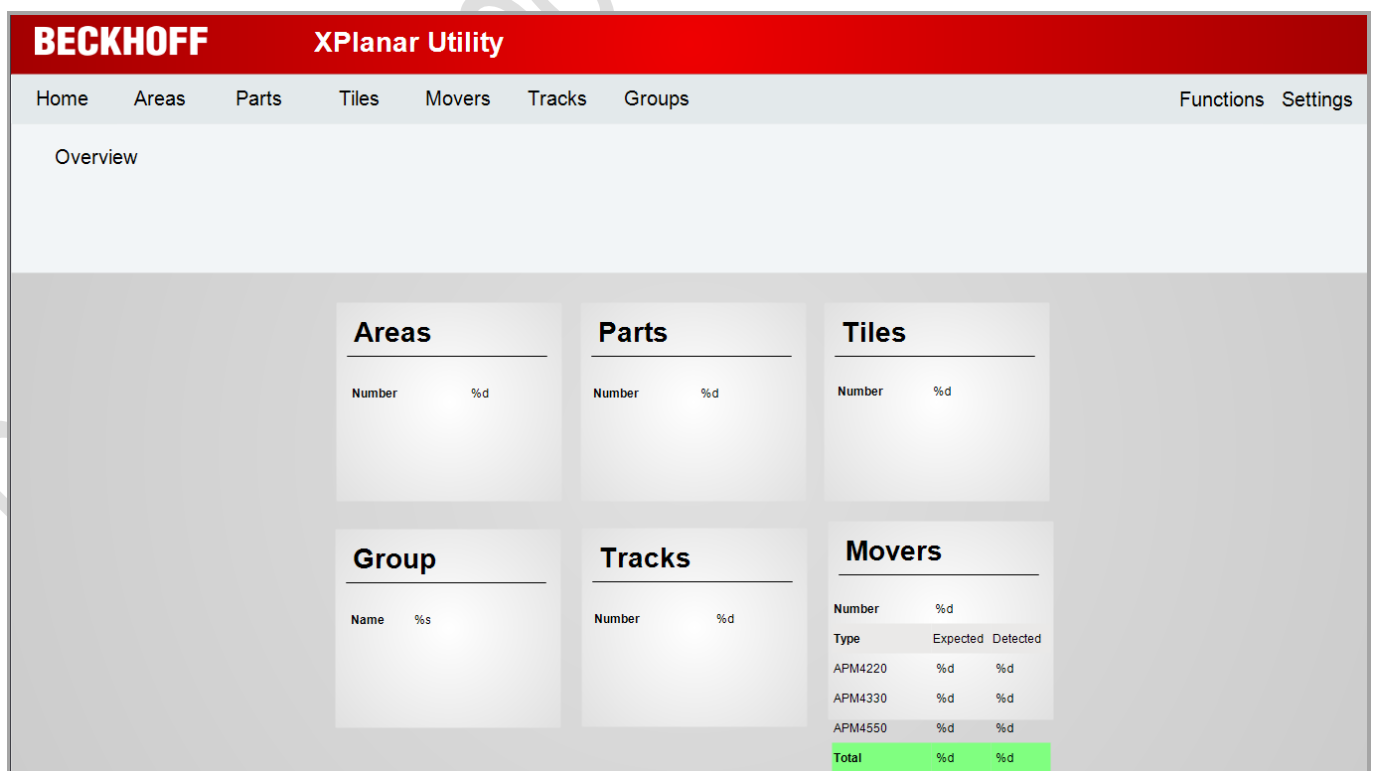


- ▶ Close the Library Repository
- ▶ References
- ▶ Add library
- ▶ Select library and add to your project

Add the Visualization to your Project



- ▶ Add a new empty Visualization Page
- ▶ Find the VI_XPlanarDiagnosis Element in your Toolbox
- ▶ Drag&Drop to your page

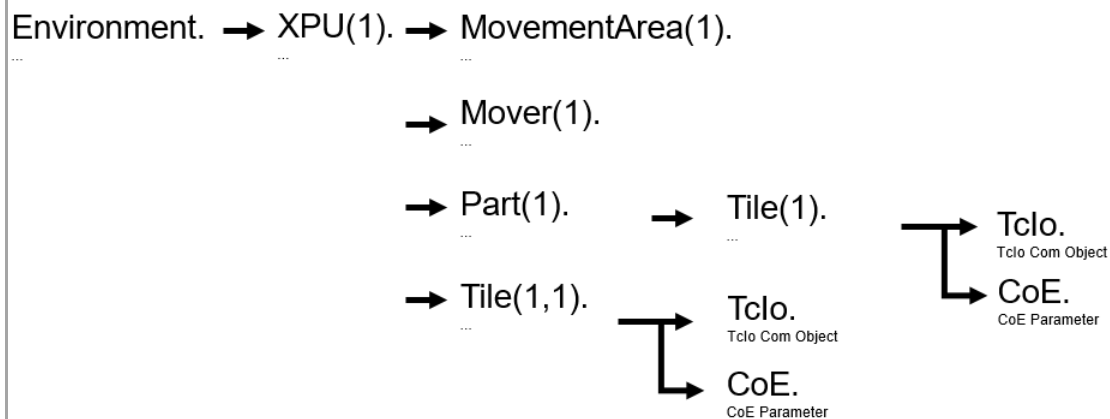


Using FB_VisuxPlanarDiag with your own HMI

DRAFT | Document in Progress

Samples

Call Chain



Lesen von Tclo Parametern

```

20:
sBtn := fbEnvironment.XpuTcIo(1).TileTcIo(1,1).GetBtn();
nStateInitXpu := 21;

```

```

sBtn := fbEnvironment.XpuTcIo(1).TileTcIo(1,1).GetBtn();
nStateInitXpu := 21;

```

Mover Parameterization Sets

```

60:
    setId := ipxpu.MoverTcIo(1).GetParameterizationSetId();
    nStateInitXpu := 61;

62:
    IF ipxpu.MoverTcIo(2).SetParameterizationSetId(SetId) THEN
        nStateInitXpu := 63;
    END_IF

```

```

60:
    setId := ipxpu.MoverTcIo(1).GetParameterizationSetId();
    nStateInitXpu := 61;

62:
    IF    ipxpu.MoverTcIo(2).SetParameterizationSetId(SetId) THEN
        nStateInitXpu := 63;
    END_IF

```

Lesen von CoE Parametern

Hinweis: Gleichzeitige CoE reads

```

18: 10:
19:   IF fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.Update() THEN
20:       fTemp1 304 := fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.TemperatureCenter;
21:       nStateInitXpu 11 := 11;
22:   END_IF

```

```

10:
    IF fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.Update() THEN
        fTemp1 := fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.TemperatureCenter;
        nStateInitXpu := 11;
    END_IF

```

Lesen eines gesamten CoE Objekts

With the Update method, the entire CoE object is read out each time. For example, if you want to read out all temperatures, the entire object can be made available with the property All.

```
stCoeDriveDiagData : POINTER TO ST_CoeDriveDiagData;
```

```
10:
  IF fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.Update() THEN
    stCoeDriveDiagData := fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.All;
    nStateInitXpu := 11;
  END_IF
```

```
10:
  IF fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.Update() THEN
    stCoeDriveDiagData := fbEnvironment.XpuTcIo(1).TileCoE(1,1).DiagData.All;
    nStateInitXpu := 11;
  END_IF
```

DcLinkOk

```
45:
IF ipXpu.TileCoE(1,1).DiagData.Update() THEN
  IF ipXpu.TileCoE(1,1).DiagData.DcLinkOk THEN
    nStateInitXpu := 46;
  END_IF
END_IF
```

```
45:
IF ipXpu.TileCoE(1,1).DiagData.Update() THEN
  IF ipXpu.TileCoE(1,1).DiagData.DcLinkOk THEN
    nStateInitXpu := 46;
  END_IF
END_IF
```

Mover Detection Setup

```
VAR
```

```
END_VAR
```

```
IF ipXpu.SetMoverDetectionSetup(aMoverDetectionSetup) THEN  
    nStateInitXpu := 31;  
END_IF
```

Start Mover Detection

```
40:  
    ipXpu.SetStartMoverDetection();  
    nStateInitXpu := 41;
```

```
40:  
    ipXpu.SetStartMoverDetection();  
    nStateInitXpu := 41;
```

Switch Controllers On/Off

```
10:  
    stTcIoControllersOn.x := TRUE;  
    stTcIoControllersOn.y := TRUE;  
    stTcIoControllersOn.z := FALSE;  
    stTcIoControllersOn.a := TRUE;  
    stTcIoControllersOn.b := TRUE;  
    stTcIoControllersOn.c := TRUE;  
  
    IF ipXpu.MoverTcIo(1).SetControllersOn(stTcIoControllersOn) THEN  
        nStateInitXpu := 11;  
    END_IF
```

```
10:
    stTcIoControllersOn.x := TRUE;
    stTcIoControllersOn.y := TRUE;
    stTcIoControllersOn.z := FALSE;
    stTcIoControllersOn.a := TRUE;
    stTcIoControllersOn.b := TRUE;
    stTcIoControllersOn.c := TRUE;

    IF ipXpu.MoverTcIo(1).SetControllersOn(stTcIoControllersOn) THEN
        nStateInitXpu := 11;
    END_IF
```

Mover Identification

Start Mover Identification

In this example, the Mover Identification is executed in the XPU. The `GetMoverIdentificationStatus()` method is used to query whether the process has been completed.

In the next step, the BTN of the individual mover can be read out on the Mover Tcio object. There is no new active reading of the BTN.

```

50:
  IF ipxpu.SetStartMoverIdentification() THEN
    nStateInitXpu 53 := 51;
  END_IF

51:
  identstatus Identifica := ipxpu.GetMoverIdentificationStatus();
  IF identstatus Identifica <> 1 THEN
    nStateInitXpu 53 := 52;
  END_IF

52:
  identifiedBtn 'test' := ipXpu.MoverTcio(1).GetIdentifiedBtn();
  nStateInitXpu 53 := 53;

```

```

50:
  IF ipxpu.SetStartMoverIdentification() THEN
    nStateInitXpu := 51;
  END_IF

51:
  identstatus := ipxpu.GetMoverIdentificationStatus();
  IF identstatus <> 1 THEN
    nStateInitXpu := 52;
  END_IF

52:
  identifiedBtn := ipXpu.MoverTcio(1).GetIdentifiedBtn();
  nStateInitXpu := 53;

```

Leave Movement Area

These are the steps to remove a Mover from a running system.

- ▶ Remove Mover from CA Group
- ▶ Disable the Mover to be removed
- ▶ Set the Movement Area to "0" on the Com Object of the Mover to be removed.

```

40:
    ipMover[1].std.Disable('');
    nstate := 41;
41:
    IF ipMover[1].std.P_Disable.Done THEN
        nstate := 42;
    END_IF
42:
    EnterOrLeaveMovementAreaParameters.MovementAreaOID := 0;
    RefEnterOrLeaveMovementAreaParameters REF= EnterOrLeaveMovementAreaParameters;

    ipXpu.MoverTcIo(1).SetMovementArea(RefEnterOrLeaveMovementAreaParameters);

    nState := 43;

```

```

VAR
    EnterOrLeaveMovementAreaParameters : EnterOrLeaveMovementAreaParameters;
    RefEnterOrLeaveMovementAreaParameters : REFERENCE TO
EnterOrLeaveMovementAreaParameters;
END_VAR

40:
    ipMover[1].std.Disable('');
    nstate := 41;
41:
    IF ipMover[1].std.P_Disable.Done THEN
        nstate := 42;
    END_IF
42:
    EnterOrLeaveMovementAreaParameters.MovementAreaOID := 0;
    RefEnterOrLeaveMovementAreaParameters REF= EnterOrLeaveMovementAreaParameters;

    ipXpu.MoverTcIo(1).SetMovementArea(RefEnterOrLeaveMovementAreaParameters);

    nState := 43;

```

Enter Movement Area

These are the steps to add a Mover to a running system.

- Place the Mover to a defined Area on a Tile
- Set the “EnterOrLeaveMovementArea” Parameter with the following information: Part or Area (OID) where the Mover is to be found and the X,Y Position referenced to the Part or Area where the Mover is to be found.
- Check if the operation is succeeded on the Mover TcIo Object “ActualPosition”.
- Enable and Add the Mover to the group again.

```

70:
  EnterOrLeaveMovementAreaParameters.MovementAreaOID := 16#01010020;
  EnterOrLeaveMovementAreaParameters.Positions.x := 52;
  EnterOrLeaveMovementAreaParameters.Positions.y := 52;

  RefEnterOrLeaveMovementAreaParameters REF= EnterOrLeaveMovementAreaParameters;

  ipXpu.MoverTcIo(1).SetMovementArea(RefEnterOrLeaveMovementAreaParameters);
  nState := 71;

71:
  PositionInfos := ipXpu.MoverTcIo(1).GetActualPositions();

  IF PositionInfos.MovementAreaOID <> 0 THEN
    nstate := 73;
  ELSE
    nstate := 70;
  END_IF

```

```

VAR
  EnterOrLeaveMovementAreaParameters : EnterOrLeaveMovementAreaParameters;
  RefEnterOrLeaveMovementAreaParameters : REFERENCE TO
EnterOrLeaveMovementAreaParameters;
END_VAR

70:
  EnterOrLeaveMovementAreaParameters.MovementAreaOID := 16#01010020;
  EnterOrLeaveMovementAreaParameters.Positions.x := 52;
  EnterOrLeaveMovementAreaParameters.Positions.y := 52;

  RefEnterOrLeaveMovementAreaParameters REF= EnterOrLeaveMovementAreaParameters;

  ipXpu.MoverTcIo(1).SetMovementArea(RefEnterOrLeaveMovementAreaParameters);
  nState := 71;

71:
  PositionInfos := ipXpu.MoverTcIo(1).GetActualPositions();

  IF PositionInfos.MovementAreaOID <> 0 THEN
    nstate := 73;
  ELSE
    nstate := 70;
  END_IF

```

DRAFT | Document in Progress

DRAFT | Document in Progress

Adressen der Mover Scope Variablen

```
5: // scope
  pInternalPositionSetpoints 16#FFFF828C1E37DA60 := ipXpu.MoverTcIo(1).ScopeAdr.GetInternalPositionSetpoints()
  nStateInitXpu 6 := 6;
```

```
VAR
    pInternalPositionSetpoints      : POINTER TO MoverVector;
END_VAR

CASE nStateInitXpu OF

5: // scope
    pInternalPositionSetpoints :=
ipXpu.MoverTcIo(1).ScopeAdr.GetInternalPositionSetpoints();
    nStateInitXpu := 6;

END_CASE
```